

Documentación Técnica: Plataforma DataForge IA

1. Introducción

El presente documento proporciona una **Documentación Técnica** exhaustiva para la plataforma **DataForge IA**, una solución web de análisis predictivo para Pequeñas y Medianas Empresas (PYMES). Su propósito es servir como una guía integral para desarrolladores, ingenieros de operaciones y personal de mantenimiento, detallando la estructura interna del sistema, la funcionalidad de sus módulos, el flujo de datos a través de la arquitectura y las tecnologías empleadas. Esta documentación es crucial para facilitar la comprensión, el desarrollo, la depuración y la evolución futura del sistema.

2. Estructura del Proyecto

El proyecto DataForge IA se organiza en un monorepositorio que contiene los microservicios del backend, la aplicación frontend y los componentes de infraestructura. La estructura de directorios principal será la siguiente:

```
dataforge-ia/
├── backend/ # Contiene los microservicios
│   ├── auth-service/ # Servicio de Autenticación y Usuarios
│   │   ├── src/
│   │   ├── tests/
│   │   └── Dockerfile
│   ├── data-management-service/ # Servicio de Gestión de Datos
│   │   ├── src/
│   │   ├── tests/
│   │   └── Dockerfile
│   ├── prediction-modeling-service/ # Servicio de Modelado Predictivo
│   │   ├── src/
│   │   ├── tests/
│   │   └── Dockerfile
│   ├── visualization-reporting-service/ # Servicio de Visualización y Reportes
│   │   ├── src/
│   │   ├── tests/
│   │   └── Dockerfile
│   └── notification-alert-service/ # Servicio de Notificaciones y Alertas
│       ├── src/
│       ├── tests/
│       └── Dockerfile
├── frontend/ # Aplicación web de interfaz de usuario
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.tsx
│   ├── tests/
│   └── package.json
├── infrastructure/ # Definiciones de infraestructura como código
│   ├── kubernetes/ # Despliegue en Kubernetes
│   ├── terraform/ # Gestión de recursos cloud
│   └── docker-compose.yml # Entorno de desarrollo local
├── docs/ # Documentación del proyecto
│   ├── architecture/
│   ├── database/
│   └── ...
├── .gitignore
├── README.md
└── package.json # Para scripts de monorepo (ej. Lerna, Nx)
```

Cada microservicio dentro del directorio `backend/` tendrá su propio `src/` para el código fuente, `tests/` para las pruebas unitarias e integración, y un `Dockerfile` para su contenedorización. El `frontend/` seguirá una estructura típica de una aplicación React/Vue.js.

3. Descripción de Módulos

Los módulos del sistema corresponden a los microservicios definidos en el diseño de arquitectura, cada uno con responsabilidades específicas y bien delimitadas:

3.1. AuthService (Servicio de Autenticación y Usuarios)

- **Tecnología Principal:** Python (Django REST Framework) / Node.js (Express.js).
- **Funcionalidad:** Gestión de usuarios (registro, login, perfil), gestión de cuentas de empresa, roles y permisos, restablecimiento de contraseñas. Emite y valida tokens JWT para la autenticación.
- **Base de Datos:** UserMetadataDB (PostgreSQL).

3.2. DataManagementService (Servicio de Gestión de Datos)

- **Tecnología Principal:** Python (Flask/FastAPI).
- **Funcionalidad:** Recibe archivos de datos (CSV, XLSX), valida su formato, realiza preprocesamiento (limpieza, transformación), mapeo de columnas, y los almacena en el `ObjectStorage`. Gestiona los metadatos de los conjuntos de datos y esquemas en `UserMetadataDB`.
- **Almacenamiento:** `ObjectStorage` (AWS S3, Google Cloud Storage).
- **Base de Datos:** `UserMetadataDB` (PostgreSQL).

3.3. PredictionModelingService (Servicio de Modelado Predictivo)

- **Tecnología Principal:** Python (Scikit-learn, TensorFlow/PyTorch).
- **Funcionalidad:** Permite la selección y configuración de modelos de IA, entrena modelos utilizando datos del `ObjectStorage` y metadatos de `UserMetadataDB`. Ejecuta predicciones y almacena los resultados en `PredictionResultsDB`. Utiliza `MessageQueue` para tareas de entrenamiento asíncronas.
- **Almacenamiento:** `ObjectStorage`.

- **Bases de Datos:** UserMetadataDB (PostgreSQL), PredictionResultsDB (MongoDB/Cassandra).
- **Comunicación Asíncrona:** MessageQueue (RabbitMQ/Kafka).

3.4. VisualizationReportingService (Servicio de Visualización y Reportes)

- **Tecnología Principal:** Node.js (Express.js) / Python (Flask) con librerías de visualización (ej. D3.js, Chart.js, Plotly).
- **Funcionalidad:** Genera paneles de control interactivos y reportes basados en los resultados de las predicciones almacenados en PredictionResultsDB y las configuraciones de UserMetadataDB. Permite la exportación a formatos como PDF y CSV.
- **Bases de Datos:** PredictionResultsDB (MongoDB/Cassandra), UserMetadataDB (PostgreSQL).

3.5. NotificationAlertService (Servicio de Notificaciones y Alertas)

- **Tecnología Principal:** Node.js (Express.js) / Python (Flask).
- **Funcionalidad:** Monitorea PredictionResultsDB para detectar condiciones de alerta predefinidas. Envía notificaciones a los usuarios a través de MessageQueue (para envío de emails, notificaciones push, etc.). Gestiona la configuración de alertas en UserMetadataDB.
- **Bases de Datos:** PredictionResultsDB (MongoDB/Cassandra), UserMetadataDB (PostgreSQL).
- **Comunicación Asíncrona:** MessageQueue (RabbitMQ/Kafka).

3.6. APIGateway (Pasarela API)

- **Tecnología Principal:** Nginx / Kong / Servicios gestionados de Cloud (AWS API Gateway).
- **Funcionalidad:** Punto de entrada unificado, enrutamiento de solicitudes, autenticación/autorización inicial, balanceo de carga, terminación SSL, gestión de políticas de tráfico y rate limiting.

4. Flujo de Datos

El flujo de datos en DataForge IA es fundamental para entender cómo la información se mueve y se transforma a través del sistema. A continuación, se describe un flujo típico para la

generación de una predicción:

1. Carga de Datos (Frontend -> APIGateway -> DataManagementService):

- El usuario, a través del **Frontend**, sube un archivo (CSV/XLSX) que es enviado al **APIGateway**.
- El **APIGateway** autentica y autoriza la solicitud, y la enruta al **DataManagementService**.
- El **DataManagementService** valida el archivo, lo preprocesa y lo almacena en el **ObjectStorage**. Los metadatos del conjunto de datos y su esquema se guardan en **UserMetadataDB**.

2. Configuración y Entrenamiento del Modelo (Frontend -> APIGateway -> PredictionModelingService -> MessageQueue):

- El usuario configura un modelo predictivo en el **Frontend**, seleccionando el tipo de modelo y los parámetros. Esta solicitud pasa por el **APIGateway** al **PredictionModelingService**.
- El **PredictionModelingService** recupera los metadatos del conjunto de datos de **UserMetadataDB** y la configuración del modelo. Envía una tarea de entrenamiento asíncrona al **MessageQueue**.
- Un worker (parte del **PredictionModelingService**) consume la tarea del **MessageQueue**, lee los datos preprocesados del **ObjectStorage**, entrena el modelo y almacena los resultados de la predicción y las métricas en **PredictionResultsDB**.

3. Visualización de Resultados (Frontend -> APIGateway -> VisualizationReportingService):

- El usuario solicita ver los resultados de una predicción en el **Frontend**. La solicitud va al **APIGateway** y luego al **VisualizationReportingService**.
- El **VisualizationReportingService** consulta **PredictionResultsDB** para obtener los resultados de la predicción y **UserMetadataDB** para la configuración del reporte.
- Genera los gráficos y el dashboard interactivo, que son enviados de vuelta al **Frontend** para su visualización.

4. Generación de Alertas (NotificationAlertService -> PredictionResultsDB -> MessageQueue):

- El **NotificationAlertService** monitorea continuamente **PredictionResultsDB** para nuevas predicciones o cambios en los resultados.
- Si se cumple una condición de alerta configurada (almacenada en **UserMetadataDB**), el **NotificationAlertService** envía un mensaje al **MessageQueue** para disparar una

notificación (ej. email).

5. Tecnologías Utilizadas

La plataforma DataForge IA se construirá utilizando un stack tecnológico moderno y robusto, seleccionado por su escalabilidad, rendimiento, madurez y soporte comunitario:

Categoría	Tecnología Principal	Propósito / Justificación
Frontend	React.js, TypeScript, Tailwind CSS	Desarrollo de interfaz de usuario interactiva, tipado estático, estilos eficientes.
Backend (Microservicios)	Python (FastAPI/Flask), Node.js (Express.js)	Python para procesamiento de datos y IA; Node.js para servicios I/O intensivos y APIs REST.
Base de Datos Relacional	PostgreSQL	Almacenamiento de datos estructurados (usuarios, metadatos), integridad transaccional.
Base de Datos NoSQL	MongoDB / Cassandra	Almacenamiento escalable de resultados de predicciones, flexibilidad de esquema.
Almacenamiento de Objetos	AWS S3 / Google Cloud Storage	Almacenamiento de datos brutos y preprocesados, alta disponibilidad y durabilidad.
Sistema de Colas de Mensajes	RabbitMQ / Apache Kafka	Comunicación asíncrona entre microservicios, manejo de tareas de larga duración.
Contenedorización	Docker	Empaquetado de microservicios para despliegue consistente.
Orquestación de Contenedores	Kubernetes	Gestión de despliegue, escalado y operación de microservicios en producción.
Modelado de IA	Scikit-learn, TensorFlow, PyTorch	Librerías estándar de la industria para desarrollo y ejecución de modelos de Machine Learning y Deep Learning.
Control de Versiones	Git	Gestión colaborativa del código fuente.
CI/CD	GitHub Actions / GitLab CI / Jenkins	Automatización de pruebas, construcción y despliegue.

Categoría	Tecnología Principal	Propósito / Justificación
Monitoreo y Logging	Prometheus, Grafana, ELK Stack	Observabilidad del sistema, recolección de métricas y logs.

6. Conclusiones

Esta documentación técnica proporciona una visión clara y detallada de la construcción de DataForge IA. La estructura modular basada en microservicios, junto con un flujo de datos bien definido y un stack tecnológico cuidadosamente seleccionado, sienta las bases para un sistema robusto, escalable y mantenible. La adhesión a estos principios y tecnologías permitirá al equipo de desarrollo construir una plataforma de análisis predictivo de alta calidad que satisfaga las necesidades de las PYMES y evolucione con el tiempo.

7. Referencias

[1] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media. [2] Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.

- Documento de Visión del Sistema: DataForge IA.
- Especificación de Requisitos de Software (SRS): DataForge IA.
- Documento de Casos de Uso: DataForge IA.
- Historias de Usuario: DataForge IA.
- Análisis de Stakeholders: DataForge IA.
- Documento de Levantamiento de Requerimientos: DataForge IA.
- Estudio de Factibilidad: DataForge IA.
- Análisis del Problema: DataForge IA.
- Diseño de Arquitectura de Software: DataForge IA.
- Diseño de Clases UML: DataForge IA.
- Diseño de Componentes: DataForge IA.
- Plan de Desarrollo Ágil: DataForge IA.
- Product Backlog: DataForge IA.
- Estándares de Desarrollo: DataForge IA.